

- Aluno: _____
- Data: 13/06/2025
- Prof.: Ricardo Couto A. da Rocha

1ª Prova de Linguagens de Programação

Aviso importantes

1. Ao final da prova, você terá que entregar todas as folhas, incluindo o enunciado.
2. As questões podem ser feitas em qualquer ordem e podem ser feitas a lápis (**exceto** questões objetivas)
3. **Todas** as questões discursivas devem ser feitas **nas folhas em anexo**.
4. **Todas** as questões objetivas devem ser marcadas a caneta no **gabarito**.

Boa prova

1ª Questão (1,5 pontos): Introdução, Legibilidade, Confiabilidade e Modelo de Execução

Considere o problema de escrever duas funções que comparem um vetor de strings:

- `strings_sao_iguais`: retorna verdadeiro se todas as strings no vetor são iguais.
- `strings_em_ordem_ascendente`: retorna verdadeiro se todas as strings no vetor estão ordenadas em ordem lexicográfica (alfabética).

Considere a solução para o problema em C do código abaixo. A função `strcmp`, da biblioteca `string.h`, compara duas strings retornando 0 se elas são iguais, um número negativo se a primeira string é “menor” lexicograficamente, e um número positivo, caso contrário.

```
1  #include <string.h>
2
3  static bool
4  strings_sao_iguais(const char **strings, int nstrings)
5  {
6      for (int i = 1; i < nstrings; i++)
7          if (strcmp(strings[0], strings[i]) != 0)
8              return false;
9      return true;
10 }
11
12 static bool
13 strings_em_ordem_ascendente(const char **strings, int nstrings)
14 {
15     for (int i = 1; i < nstrings; i++)
16         if (strcmp(strings[i - 1], strings[i]) >= 0)
17             return false;
18     return true;
19 }
```

Considere uma implementação equivalente dessas duas funções em Python.

1.1 - Legibilidade e Confiabilidade

Compare as implementações em C e em Python em termos de (1) legibilidade e (2) confiabilidade, considerando os aspectos das linguagens (exibidos nos códigos C). Se necessário, coloque um trecho de código em Python para justificar suas afirmativas.

RESPOSTA ESPERADA:

1. Legibilidade: a operação de comparação de Strings em C com `strcmp` não é explícita no tipo de comparação a ser feito e no seu significado. Infelizmente, o programador em C está preso ao uso de `strcmp` o que não representa adequadamente ou de maneira mais legível a simples comparação entre duas strings.

2. Confiabilidade: C apresenta aspectos que tornam o código menos confiável, se comparado com Python, como exigir que o usuário da função indique explicitamente o tamanho dos vetores de Strings e permite tentativa de acesso a regiões de memória para além das strings e dos vetores

1.2 - Modelo de Execução

Discuta **se** e **como** o modelo de execução dessas duas linguagens interfere na (1) legibilidade e (2) confiabilidade desses mesmos códigos para cada uma das linguagens.

RESPOSTA ESPERADA:

1. Legibilidade: não interfere diretamente a legibilidade de nenhuma das linguagens.
2. Confiabilidade: o modelo de execução de C - linguagem compilada - garante que problemas de incompatibilidade de tipos e invocações inconsistentes serão detectados em tempo de compilação e devem ser resolvidos antes da execução, o que confere maior confiabilidade ao código (especificamente nesse aspecto).

2ª Questão (1,5 pontos): Sintaxe e Semântica

2.1 - Sintaxe com BNF

Considere ****** como o operador de potenciação de uma certa linguagem hipotética para descrever expressões como 2^{3^0} . Fazendo o **uso de BNF**, descreva a sintaxe de uma expressão de potenciação considerando que nessa linguagem o operador de potenciação é **associado à direita**. Crie os tokens que julgar necessário, mas a sua sintaxe só precisa descrever a estrutura de uma expressão com o operador como, por exemplo,

```
2 ** 3 ** 1
```

RESPOSTA ESPERADA:

```
<expressao-potencia> <- <numero> ** <expressao-potencia>
<expressao-potencia> <- <numero> ** <numero>

<expressao-potencia> <- <numero> ** <expressao-potencia>
<expressao-potencia> <- <numero> ** <numero>
```

2.2 - Gramática de Atributos

RESPOSTA ESPERADA/SUGERIDA:

Para simplificar a resposta, considerarei que:

- a sintaxe BNF da questão anterior não será modificada, e então o **<numero>** pode ser tanto uma string como um número, descrito aqui como um tipo **numeral** (inteiro ou real).
- a sintaxe BNF também não será modificada para permitir a composição variáveis, mas apenas expressões literais expressas em **<numero>**.

- 1) Regra Sintática: **<expressao-potencia> -> <numero>[2] ** <expressao-potencia>[3]**
- 2) Regra Sintática: **<expressao-potencia> <- <numero>[2] ** <numero>[3]**
- 3) Regra Semântica: **<expressao-potencia>.actual_type <- {string | numeral}**
- 4) Regra Semântica: **<expressao-potencia>.actual_type <-**
 if (<numero>[2].actual_type == string) and (<expressao-potencia>[3].actual_type == string)
 return string
 else if (<numero>[2].actual_type == numeral) and (<expressao-potencia>[3].actual_type == numeral)
 return numeral
- 5) Predicado: **(<numero>[2].actual_type == string) <=> (<expressao-potencia>[3].actual_type == string)**
- 6) Predicado: **(<numero>[2].actual_type == string) <=> (<numero>[3].actual_type == string)**

3ª Questão (1,5 pontos): Nomes, Vinculação, Verificação de Tipos e Escopo

Considere o problema de implementar uma função **split** que recebe uma string e retorna uma string com os caracteres da string original com o caracter “,” (e espaço) delimitando sempre que houver uma mudança no caracter na string. Assim, a invocação

```
split("gHHH5YY++///x")
```

deveria retornar a string

```
"g, HHH, 5, YY, ++, ///, x"
```

Considere agora as duas implementações de **split** em C e em Python abaixo.

```

1 // Implementação em C
2 char *split(char *str)
3 {
4     char ultimo = *str;
5     char *resultado = malloc(3*strlen(str));
6     char *contador = resultado;
7     for (char *pos_c = str; *pos_c != '\0'; pos_c++) {
8         if (*pos_c != ultimo) {
9             strcpy(contador, ", ");
10            contador = contador + 2;
11            ultimo = *pos_c;
12        }
13        *contador = *pos_c;
14        contador++;
15    }
16    *(contador--)= '\0';
17    return realloc(resultado, strlen(resultado) );
18 }

```

Para a implementação em C:

- A função `strlen` retorna o tamanho de uma string terminada em nulo
- A função `realloc` muda o tamanho do espaço em memória ocupado por um vetor para o tamanho indicado, fazendo uma alocação em uma nova área de memória se necessário e copiando o conteúdo do vetor para a nova área. O ponteiro retornado pode ser no mesmo endereço ou não daquele passado como parâmetro, dependendo se uma nova alocação ocorre.

```

1  # Implementação em Python
2  def split(str):
3      ultimo = str[0]
4      resultado = [ultimo]
5      for pos_c in range(1, len(str)):
6          if str[pos_c] != ultimo:
7              resultado = resultado + ["", ""]
8              ultimo = str[pos_c]
9
10         resultado = resultado + [str[pos_c]]
11
12     return ''.join(resultado)

```

3.1 - Amarrações em Linguagens

Indique (1) **quando** e (2) **onde** ocorre a amarração de armazenagem das variáveis abaixo nos dois programas. Por exemplo, o momento da amarração pode ser compilação, execução, carga ou outro momento.

- str
- ultimo
- resultado
- contador
- pos_c

RESPOSTA ESPERADA

Variável	Amarr. Armazenagem (Python)	Amarr. Armazenagem (C)	Local (Python)	Local (C)
str	Dinâmica - execução	Dinâmica - execução	Pilha	Pilha
ultimo	Dinâmica - execução	Dinâmica - execução	Pilha	Pilha
resultado	Dinâmica - execução	Dinâmica - execução	Pilha/Heap	Heap
contador	Dinâmica - execução	Dinâmica - execução	Pilha	Pilha
pos_c	Dinâmica - execução	Dinâmica - execução	Pilha	Pilha

3.2 - Escopo Estático e Dinâmico

Considere o seguinte trecho de código de uma linguagem hipotética.

```

1  int a = 2
2  int b = 4
3
4  int funcaoA() {
5      int a = 8
6      return funcaoB()
7  }
8
9  int funcaoB() {
10     int b = 16
11     return funcaoAB()
12 }
13
14 int funcaoAB() {
15     return a + b
16 }

```

Quais serão os valores retornados para as chamadas `funcaoA()`, `funcaoB()` e `funcaoAB()` caso a linguagem utilize:

a. Escopo Dinâmico

- **RESPOSTA ESPERADA:** 24 (8+16), 18 (2+16), 6

b. Escopo Estático

- **RESPOSTA ESPERADA:** 6, 6, 6

4ª Questão (1,5 pontos): Tipos de Dados

Para as questões abaixo, considere que um programador precisa armazenar uma tabela com a lista de salas de um prédio e os alunos presentes atualmente em cada sala. Ambas as informações serão descritas como strings conforme indicado no exemplo abaixo.

Sala 301	Sala 302	Sala 303
Clara	Ana	Daniel
Henrique	Letícia	Renata
José	Rafael	Rodrigo

4.1 - Tipos Primitivos e Compostos

Compare o armazenamento da tabela utilizando os tipos primitivos ou comumente oferecidos nas linguagens **C** e **Python**.

Compare tanto a instanciação do dado como a dinamicidade (possibilidade de alteração) do dado para, por exemplo, acrescentar/remover/modificar os alunos ou salas.

RESPOSTA ESPERADA

- **Python:**
 - Pode-se utilizar um vetor de vetores, utilizando lista (`list`) de listas de Python.
 - Como listas são dinâmicas em Python e a memória é gerenciada automaticamente, não haveria nenhuma dificuldade em gerenciar a alteração da tabela.
- **C:**
 - a representação como matriz ou vetor de vetores é problemática pois em C eles os vetores são estáticos (não podem mudar de tamanho).
 - Considerando que o número de salas é fixo, então seria necessário pré-alocar as salas com o máximo possível de entradas e controlar o uso indicando a pessoa em cada sala.
 - Em cada posição, haveria um ponteiro para caracter (`char*`) que poderia ser alocado e desalocado da heap sem qualquer problema no gerenciamento da memória. O ponteiro nulo (`NULL`) poderia ser utilizado para indicar a partir de qual posição não há mais alunos em posição do vetor/matriz.
 - É adequado usar um vetor de vetores, pois permite deixar fixo o número de salas e ajustar apenas o número de alunos no vetor associado a cada sala.

4.2 - Implementação de Arrays em C

Para o mesmo cenário, crie um exemplo **na linguagem C** de uma tabela com duas salas e dois alunos em cada sala (utilize nomes curtos). Mostre a tabela representada e a representação em memória dos dados para os exemplos criados.

RESPOSTA ESPERADA

```
1  int numero_salas = 20
2  int max_alunos = 55 // não utilizado no exemplo
3
4  char* sala301[] = {"Sala 301", "Clara", "Henrique"};
5  char* sala302[] = {"Sala 302", "Ana", "Rafael"};
6  char** tabela[] = malloc(sizeof(char**) * numero_salas);
7  tabela[0] = sala301;
8  tabela[1] = sala302;
9
10 /* idealmente, código deveria inicializar outras
11    posições de tabela[] bem como inicializar os vetores de sala
12    com o max_alunos posições (inicializadas com NULL a partir do
13    índice 3)
14 */
```

5ª Questão (4,0 pontos): Questões Objetivas

Para cada uma das questões abaixo assinale a **única** alternativa adequada à pergunta formulada.

5.1 - Legibilidade, Ortogonalidade, Confiabilidade e Redigibilidade

Em Python, as variáveis não possuem um tipo pré-definido, e assumem o tipo do último dado nela armazenado. Assim, na atribuição `a = []` seguida da atribuição `a = 23`, a variável `a` é uma lista na primeira, enquanto torna-se inteiro na segunda. Essa decisão no projeto da linguagem impacta o código

- a. ☐ diminuindo a redigibilidade
- b. ☒ diminuindo a confiabilidade
- c. ☐ diminuindo a ortogonalidade
- d. ☐ aumentando a ortogonalidade
- e. ☐ aumentando a legibilidade
- f. ☐ aumentando o desempenho

5.2 - Modelos de Execução

Scala é uma linguagem funcional cujo código é compilado em bytecodes Java. Sabendo unicamente disso, assinale a afirmativa verdadeira.

- a. ☐ Um código Scala executará mais rápido que o equivalente código Java.
- b. ☐ Um código Scala será mais lento que o equivalente código Java.
- c. ☒ O modelo de execução de Scala é o mesmo de Java.
- d. ☐ O modelo de execução de Scala é de linguagem interpretada.
- e. ☐ Scala possui a mesma sintaxe que Java.
- f. ☐ Scala possui a mesma semântica que Java.

5.3 - Sintaxe

A sintaxe

- a. ☒ Define a estrutura de todas as unidades de um programa.
- b. ☐ Define o comportamento das unidades de um programa.
- c. ☐ Define o modelo de execução de um programa para uma certa linguagem.
- d. ☐ Define o paradigma da linguagem de programação.
- e. ☐ Define a legibilidade de uma linguagem de programação.
- f. ☐ Define a confiabilidade de uma linguagem de programação.

5.4 - Semântica

Qual das regras abaixo **NÃO** pode ser especificada em BNF, ou seja, em uma gramática livre de contexto?

- a. ☐ Em Java, todo código precisa iniciar com uma lista de cláusulas `import`.
- b. ☐ Em Java, a declaração de um método pode possuir um modificador de visibilidade `public`, `protected`, `packaged` ou nenhum.
- c. ☐ Em C, os índices não podem ser inicializados na declaração do `for` (exemplo: `for(int i = 0; ...)`) como em Java.
- d. ☒ Em C, não podem ser declaradas duas variáveis com o mesmo nome.
- e. ☐ Em Python uma sentença termina unicamente com o início de uma nova linha (não há uso de `;` como em C).

- f. ☐ NENHUMA das demais regras pode ser especificadas com BNF.

5.5 - Amarrações

Em **Java**, quando ocorre a amarração sobre o endereço de uma variável local a uma função?

- a. ☐ Na definição da linguagem
- b. ☐ Na implementação da linguagem
- c. ☐ Na compilação do programa
- d. ☐ Na ligação do programa (link)
- e. ☐ Na carga do programa
- f. ☒ Na execução do programa

5.6 - Valor-l e Valor-r

No código Python abaixo é possível afirmar

```
a = 0
b = a
```

- a. ☒ As variáveis possuem o mesmo `valor_r`, mas `valor_l` diferente.
- b. ☐ As variáveis possuem o mesmo `valor_l`, mas `valor_r` diferentes.
- c. ☐ As variáveis possuem o mesmo `valor_l` e `valor_r`.
- d. ☐ As variáveis possuem `valor_r` e `valor_l` diferentes.

5.7 - Heap, Pilha, Estático

Assinale a afirmativa verdadeira.

- a. ☐ Uma função não pode/deve retornar o endereço de uma variável na pilha.
- b. ☐ Uma função não pode/deve retornar o endereço de uma variável na heap criada por ela, quando o heap é explícito.
- c. ☐ Uma variável de pilha se mantém entre duas invocações de uma mesma função.
- d. ☐ Uma variável de heap explícito, criada dentro de uma função, deixa de existir quando a função termina.
- e. ☐ Para diversas invocações de `malloc` deve existir exatamente uma única invocação de `free`.
- f. ☐ [XXX] Linguagens que utilizam armazenamento em heap implícito deve utilizar coleta de lixo (*garbage collector*).

5.8 - Escopo

Considere os dois códigos C e Java, *aparentemente* equivalentes. A saída do código C é

```
6: 1
8: 2
10: 0
10: 1
12: 2

1  #include <stdio.h>
2
3  int c = 1;
4
5  void main() {
6      printf("6: %d\n", c);
7      int c = 2;
8      printf("8: %d\n", c);
9      for (int c = 0; c < 2; c++) {
10         printf("10: %d\n", c);
11     }
12     printf("12: %d\n", c);
13 }
```

Por sua vez, no código Java, a variável `c` na linha 9 (laço `for`) não pode ser declarada como inteiro, pois Java não permite que laços `for` estabeleçam novos escopos de variáveis. Caso deixado como o código C, o código Java não compila, produzindo a mensagem de erro “*variável c já está definida no método main(String[])*”.

```
1  public class Escopo {
2
3      public static int c = 1;
4
5      public static void main(String[] args) {
6          System.out.println("6: " + c);
7          int c = 2;
8          System.out.println("8: " + c);
9          for (c = 0; c < 2; c++) {
10             System.out.println("10: " + c);
11         }
12         System.out.println("12: " + c);
13     }
```

```
14
15 }
```

Assinale a afirmativa verdadeira.

- a. ☐ O código Java produzirá uma saída diferente do código C.
- b. ☐ Todas as variáveis `c` em ambos os códigos referenciam a mesma variável em memória.
- c. ☐ [XXX] A variável `c` na linha 6 dos códigos é resolvida estaticamente.
- d. ☐ A variável `c` na linha 8 dos códigos é resolvida dinamicamente.
- e. ☐ A variável `c` na linha 10 do código Java é resolvida dinamicamente.
- f. ☐ A linha 12 do código Java escreverá “12: 2”, tal qual o código C.

5.9 - Tipos

O objetivo de um tipo ordinal ou enumeração, tal como `enum` da linguagem C, é

- a. ☐ Garantir em tempo de compilação que os valores de um tipo de variável são sempre inicializados.
- b. ☐ Garantir em tempo de execução que os valores de um tipo de variável não extrapolam os limites de um tipo.
- c. ☐ Controlar o valor máximo de um dado numérico em tempo de compilação.
- d. ☐ Controlar quando atribuições numéricas a variáveis são permitidas em linguagens dinâmicas, como Python.
- e. ☐ [XXX] Permitir descrever um dado limitado a um conjunto particular de valores.
- f. ☐ Permitir a um dado de certo tipo descrever valores para além dos limites pré-estabelecidos no tipo.

5.10 - Tipos

Considere o seguinte código Python:

```
1  a = "----"
2  print(a)
3  b = a
4  print(b)
5  a = "xxxx"
6  print(a)
7  print(b)
```

A execução deste código gerou a saída abaixo, como resultado do comando `print` nas linhas 2, 4, 5 e 6.

```
----
----
xxxx
----
```

Baseando unicamente no comportamento deste código é possível afirmar que

- a. ☐ o tipo `string` em Python é imutável.
- b. ☐ [XXX] o código não é suficiente para afirmar que o tipo `string` é imutável.
- c. ☐ se o tipo `string` for imutável, então o código da linha (5) causará um erro de execução.
- d. ☐ as duas últimas linhas na saída do código mostram que o tipo `string` é mutável.

5.11 - Arrays

Na implementacao arrays (vetores) na linguagem C, para um vetor `v` que armazena dados do tipo `tipo_item`, o elemento `v[i]` (`i` é inteiro) está no endereço

- a. [] `addr(v[0]) + i`
- b. [] `addr(tipo_item) + i`
- c. [XXX] `addr(v[0]) + i*sizeof(tipo_item)`
- d. [] `i*addr(v[0]) + sizeof(tipo_item)`
- e. [] `addr(v[0]) + sizeof(tipo_item)`
- f. [] `addr(i*sizeof(tipo_item)) + v[0]`

5.12 - Registros

No registro (`struct`) C abaixo, considerando que o tipo `char` ocupa 1 bytes e o tipo `int` ocupa 4 bytes, o valor do campo `b` estará no endereço de memória de `var_s` acrescido de

```
1  typedef struct {
2      char a[5];
3      int b;
4      int c;
5  } Estrutura;
6
7  Estrutura var_s;
```

- a. [] 1 byte
- b. [] 4 bytes
- c. [XXX] 5 bytes
- d. [] 6 bytes
- e. [] 8 bytes
- f. [] 9 bytes